



The cost of computation and supercomputers

FLOPs, GPU hours, dollars and energy

Carlos Cotrini



The same three questions, asked for the AI

Was kann *es* wissen? → Weekend 5: RAG

Was soll *es* tun? → Weekend 6: Agentic AI

→



The same three questions, asked for the AI

Was kann *es* wissen? → Weekend 5: RAG

Was soll *es* tun? → Weekend 6: Agentic AI

Was darf *es* hoffen? → Weekend 7: Large Scale AI what it may become, for what we spend



The same three questions, asked for the AI

Was kann *es* wissen? → Weekend 5: RAG

Was soll *es* tun? → Weekend 6: Agentic AI

Was darf *es* hoffen? → Weekend 7: Large Scale AI what it may become, for what we spend

Weekend 5 was what it can know, Weekend 6 what it should do. And this weekend is what it may become.



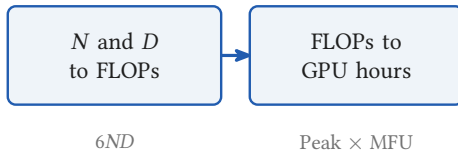
What the next hour computes

N and D
to FLOPs

$$6ND$$

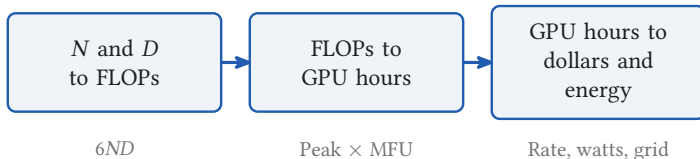


What the next hour computes





What the next hour computes





Section 1

Dividing a computation, 1793



Dividing a computation, 1793

Adam Smith, 1776



The pin maker's workshop. *Epinglier*, Diderot and d'Alembert's

Encyclopedie, 1762

Adam Smith, 1776



The pin maker's workshop. *Epinglier*, Diderot and d'Alembert's

Encyclopedie, 1762

One man draws out the wire, another straightens it, a third cuts it, a fourth points it, a fifth grinds it at the top for receiving the head.

Adam Smith, 1776



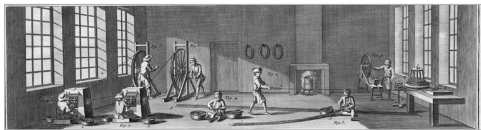
The pin maker's workshop. *Epinglier*, Diderot and d'Alembert's

Encyclopedie, 1762

One man draws out the wire, another straightens it, a third cuts it, a fourth points it, a fifth grinds it at the top for receiving the head.

- *The Wealth of Nations*, Book I, chapter 1: **Of the Division of Labour**

Adam Smith, 1776



The pin maker's workshop. *Epinglier*, Diderot and d'Alembert's

Encyclopedie, 1762

One man draws out the wire, another straightens it, a third cuts it, a fourth points it, a fifth grinds it at the top for receiving the head.

- *The Wealth of Nations*, Book I, chapter 1: **Of the Division of Labour**
- One workman making a whole pin: perhaps twenty in a day, perhaps not one

Adam Smith, 1776



The pin maker's workshop. *Epinglier*, Diderot and d'Alembert's

Encyclopedie, 1762

One man draws out the wire, another straightens it, a third cuts it, a fourth points it, a fifth grinds it at the top for receiving the head.

- *The Wealth of Nations*, Book I, chapter 1: **Of the Division of Labour**
- One workman making a whole pin: perhaps twenty in a day, perhaps not one
- Ten men, split across eighteen operations: **forty eight thousand**

Adam Smith, 1776



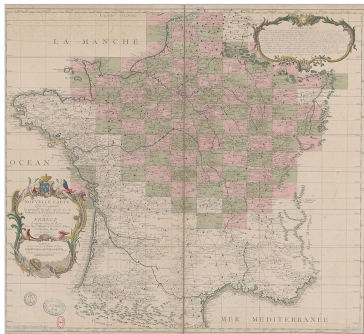
The pin maker's workshop. *Epinglier*, Diderot and d'Alembert's

Encyclopedie, 1762

One man draws out the wire, another straightens it, a third cuts it, a fourth points it, a fifth grinds it at the top for receiving the head.

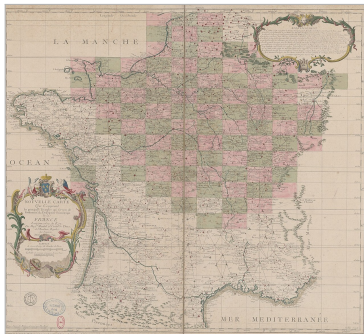
- *The Wealth of Nations*, Book I, chapter 1: **Of the Division of Labour**
- One workman making a whole pin: perhaps twenty in a day, perhaps not one
- Ten men, split across eighteen operations: **forty eight thousand**
- Same people, same tools. Only the organisation changed

The task: the area of every parcel in France



The principal triangles of France. Maraldi and Cassini de Thury, 1744

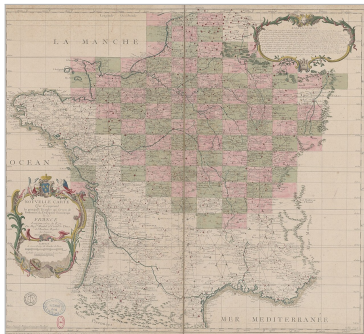
The task: the area of every parcel in France



The principal triangles of France. Maraldi and Cassini de Thury, 1744

- The Revolution replaces tax privilege with one tax on land

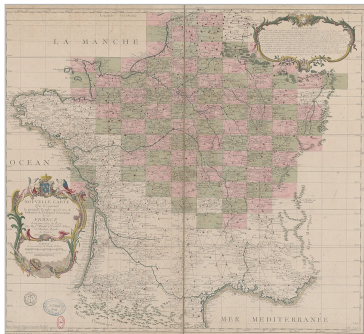
The task: the area of every parcel in France



The principal triangles of France. Maraldi and Cassini de Thury, 1744

- The Revolution replaces tax privilege with one tax on land
- So every parcel has to be measured, and its area computed

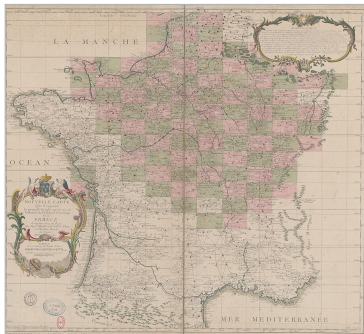
The task: the area of every parcel in France



The principal triangles of France. Maraldi and Cassini de Thury, 1744

- The Revolution replaces tax privilege with one tax on land
- So every parcel has to be measured, and its area computed
- Gaspard de Prony directs the Bureau du Cadastre, which does the computing

The task: the area of every parcel in France



The principal triangles of France. Maraldi and Cassini de Thury, 1744

- The Revolution replaces tax privilege with one tax on land
- So every parcel has to be measured, and its area computed
- Gaspard de Prony directs the Bureau du Cadastre, which does the computing
- Land is measured in triangles, because angles are quick and chains are slow

Three tiers, and what each hands down

Five or six mathematicians
Choose the formulas and the blocks

Three tiers, and what each hands down

Five or six mathematicians

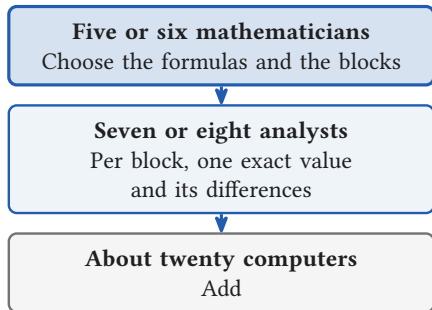
Choose the formulas and the blocks



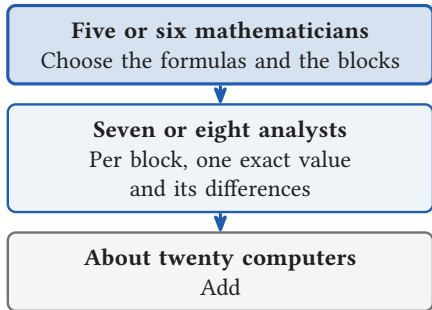
Seven or eight analysts

Per block, one exact value
and its differences

Three tiers, and what each hands down

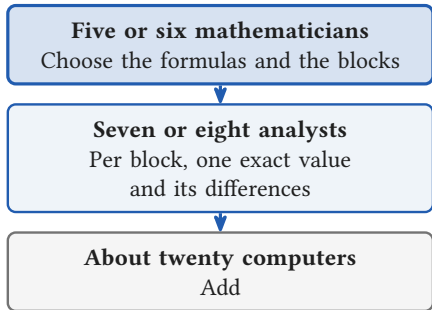


Three tiers, and what each hands down



Je conçus tout à coup l'idée d'appliquer la même méthode à l'immense travail dont je m'étais laissé imposer le fardeau, et de fabriquer mes logarithmes comme on fabrique des épingles.

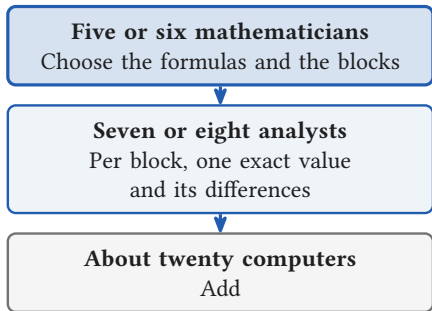
Three tiers, and what each hands down



Je conçus tout à coup l'idée d'appliquer la même méthode à l'immense travail dont je m'étais laissé imposer le fardeau, et de fabriquer mes logarithmes comme on fabrique des épingles.

Manufacturing my logarithms as one manufactures pins.
Gaspard de Prony, Notices, 1824, p. 5.

Three tiers, and what each hands down

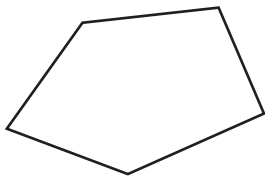


Je conçus tout à coup l'idée d'appliquer la même méthode à l'immense travail dont je m'étais laissé imposer le fardeau, et de fabriquer mes logarithmes comme on fabrique des épingles.

Manufacturing my logarithms as one manufactures pins.
Gaspard de Prony, Notices, 1824, p. 5.

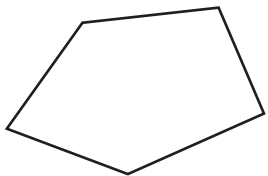
Smith's ten workmen each did a different step. Prony's twenty clerks all did the same step, on different blocks. The third tier is data parallel.

One parcel, many triangles, one each

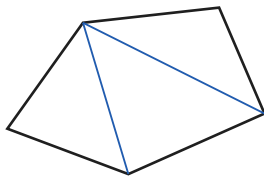


One parcel

One parcel, many triangles, one each

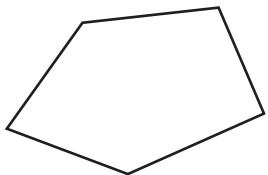


One parcel

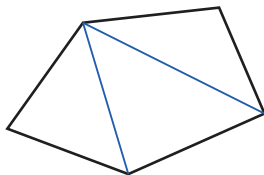


Cut into triangles

One parcel, many triangles, one each



One parcel

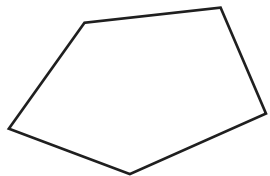


Cut into triangles

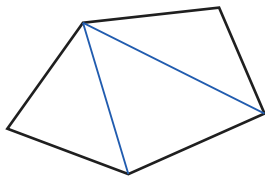


One each

One parcel, many triangles, one each



One parcel



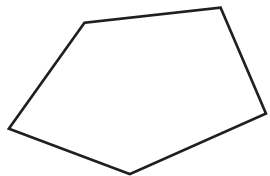
Cut into triangles



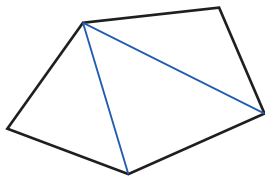
One each

- A clerk never sees a parcel. He is handed one triangle and returns one area

One parcel, many triangles, one each



One parcel



Cut into triangles



One each

- A clerk never sees a parcel. He is handed one triangle and returns one area
- Which is why twenty of them can work at once without talking to each other

One side, two angles, and a table of logarithms

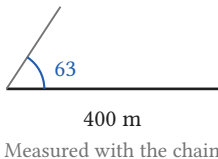
$$\text{Area} = \frac{c^2 \sin A \sin B}{2 \sin C}$$

400 m

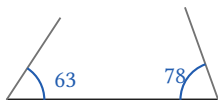
Measured with the chain

One side, two angles, and a table of logarithms

$$\text{Area} = \frac{c^2 \sin A \sin B}{2 \sin C}$$



One side, two angles, and a table of logarithms

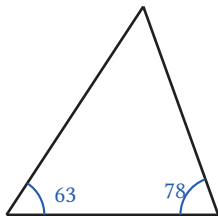


400 m

Measured with the chain

$$\text{Area} = \frac{c^2 \sin A \sin B}{2 \sin C}$$

One side, two angles, and a table of logarithms

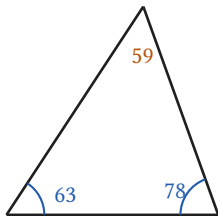


400 m

Measured with the chain

$$\text{Area} = \frac{c^2 \sin A \sin B}{2 \sin C}$$

One side, two angles, and a table of logarithms



400 m

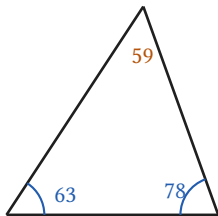
Measured with the chain

The third angle is free:

$200 - 63 - 78$

$$\text{Area} = \frac{c^2 \sin A \sin B}{2 \sin C}$$

One side, two angles, and a table of logarithms



400 m

Measured with the chain

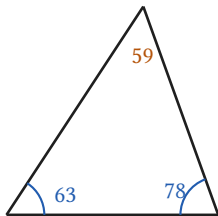
The third angle is free:

200 - 63 - 78

$$\text{Area} = \frac{c^2 \sin A \sin B}{2 \sin C}$$

By hand that is a square, two products and a division

One side, two angles, and a table of logarithms



400 m

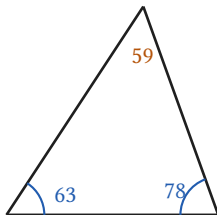
Measured with the chain

The third angle is free:

$$200 - 63 - 78$$

$$\log \text{Area} = 2 \log c + \log \sin A + \log \sin B \\ - \log \sin C - \log 2$$

One side, two angles, and a table of logarithms



400 m

Measured with the chain

The third angle is free:

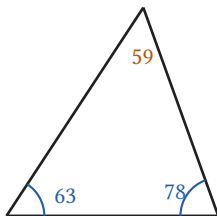
200 — 63 — 78

$$\log \text{Area} = 2 \log c + \log \sin A + \log \sin B \\ - \log \sin C - \log 2$$

$2 \log 400$	$+5.2041200$
$+ \log \sin 63$	-0.0778938
$+ \log \sin 78$	-0.0264654
$- \log \sin 59$	-0.0970812
$- \log 2$	$+0.3010300$

Five lookups in a seven figure table, then four additions. The parcel is ours; the units and the method are the period's.

One side, two angles, and a table of logarithms



400 m

Measured with the chain

The third angle is free:

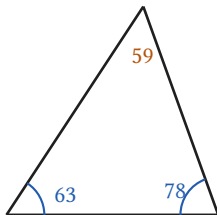
200 — 63 — 78

$$\log \text{Area} = 2 \log c + \log \sin A + \log \sin B \\ - \log \sin C - \log 2$$

$2 \log 400$	+5.2041200
+ $\log \sin 63$	−0.0778938
+ $\log \sin 78$	−0.0264654
− $\log \sin 59$	−0.0970812
− $\log 2$	+0.3010300
$\log \text{Area}$	4.8958120

Five lookups in a seven figure table, then four additions. The parcel is ours; the units and the method are the period's.

One side, two angles, and a table of logarithms



400 m

Measured with the chain

The third angle is free:

$200 - 63 - 78$

$$\log \text{Area} = 2 \log c + \log \sin A + \log \sin B - \log \sin C - \log 2$$

$2 \log 400$	+5.2041200
+ $\log \sin 63$	−0.0778938
+ $\log \sin 78$	−0.0264654
− $\log \sin 59$	−0.0970812
− $\log 2$	+0.3010300

$\log \text{Area}$	4.8958120
--------------------	-----------

Area	78 670 m ²	about 7.9 ha
------	-----------------------	--------------

Five lookups in a seven figure table, then four additions. The parcel is ours; the units and the method are the period's.



Section 2

One operation, and who does it

2

☰ One operation, and who does it

A FLOP is one operation on two numbers

One multiply accumulate

☰ One operation, and who does it

A FLOP is one operation on two numbers

One multiply accumulate

$a \times b$
one multiply

☰ One operation, and who does it

A FLOP is one operation on two numbers

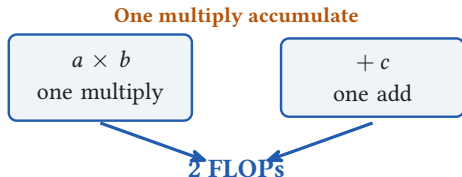
One multiply accumulate

$a \times b$
one multiply

$+ c$
one add

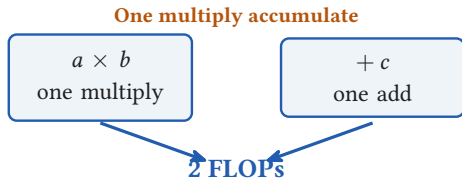
☰ One operation, and who does it

A FLOP is one operation on two numbers



☰ One operation, and who does it

A FLOP is one operation on two numbers

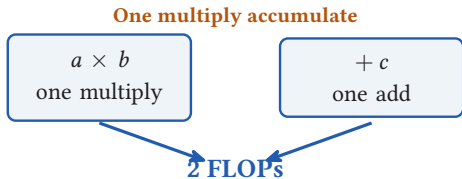


Count, or rate

FLOP: a count of operations
how much work

☰ One operation, and who does it

A FLOP is one operation on two numbers



Count, or rate

FLOP: a count of operations
how much work

FLOP/s: operations per second
how fast

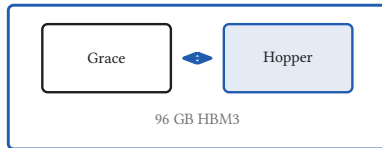
☰ One operation, and who does it

One computer, then and now



A computer room, NACA, 1949

Human computer, 1949



One GH200. Alps holds 10,752

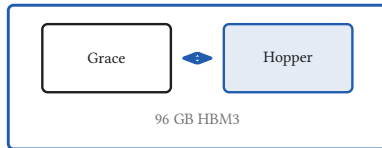
One GH200, 2024

☰ One operation, and who does it

One computer, then and now



A computer room, NACA, 1949



One GH200. Alps holds 10,752

Human computer, 1949

One GH200, 2024

Rate

1,000 additions a day

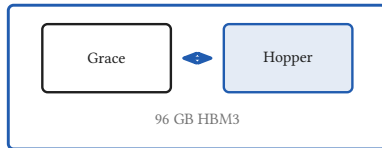
990 TFLOP/s dense BF16

One operation, and who does it

One computer, then and now



A computer room, NACA, 1949



One GH200. Alps holds 10,752

Human computer, 1949

Rate
Format

1,000 additions a day
Decimal, by hand

One GH200, 2024

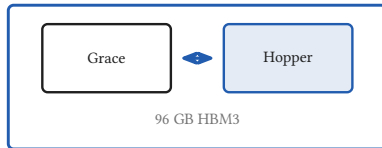
990 TFLOP/s dense BF16
16 bits, fixed

One operation, and who does it

One computer, then and now



A computer room, NACA, 1949



One GH200. Alps holds 10,752

Human computer, 1949

Rate	1,000 additions a day
Format	Decimal, by hand
Power	About 100 W

One GH200, 2024

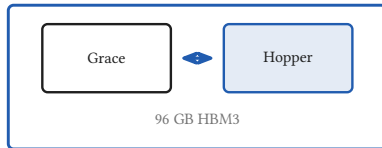
990 TFLOP/s dense BF16
16 bits, fixed
660 W on Alps

One operation, and who does it

One computer, then and now



A computer room, NACA, 1949



One GH200. Alps holds 10,752

Human computer, 1949

Rate	1,000 additions a day
Format	Decimal, by hand
Power	About 100 W
Food	2,065 kcal a day what one person eats

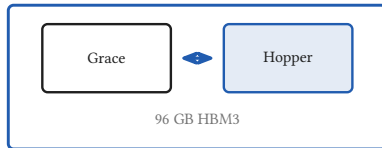
One GH200, 2024

990 TFLOP/s dense BF16
16 bits, fixed
660 W on Alps
13,600 kcal a day what six or seven people eat

One computer, then and now



A computer room, NACA, 1949



One GH200. Alps holds 10,752

Human computer, 1949

Rate	1,000 additions a day
Format	Decimal, by hand
Power	About 100 W
Food	2,065 kcal a day
	what one person eats

One GH200, 2024

990 TFLOP/s dense BF16
16 bits, fixed
660 W on Alps
13,600 kcal a day
what six or seven people eat

- The clerk's rate is what the shop *delivered*. The chip's is what it *could* do

☰ One operation, and who does it

A GH200 is a CPU and a GPU on one module



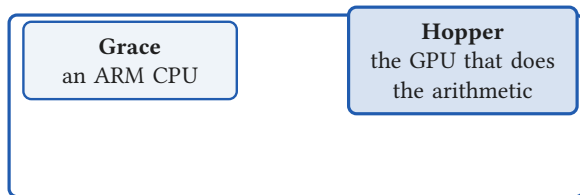
☰ One operation, and who does it

A GH200 is a CPU and a GPU on one module



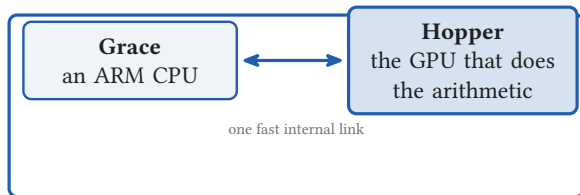
☰ One operation, and who does it

A GH200 is a CPU and a GPU on one module



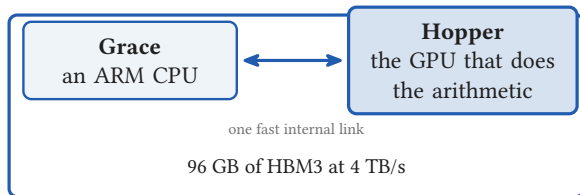
☰ One operation, and who does it

A GH200 is a CPU and a GPU on one module

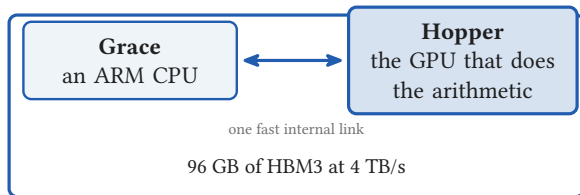


☰ One operation, and who does it

A GH200 is a CPU and a GPU on one module

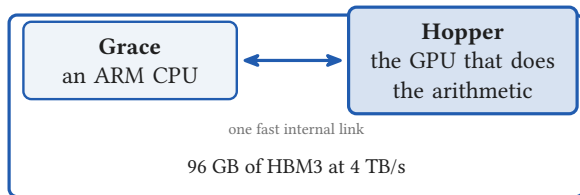


A GH200 is a CPU and a GPU on one module



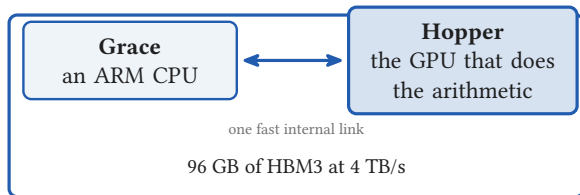
- The GPU computes. The CPU reads the data, runs Python, and keeps it fed

A GH200 is a CPU and a GPU on one module



- The GPU computes. The CPU reads the data, runs Python, and keeps it fed
- HBM3 is memory stacked on the GPU package: only 96 GB, but 4 TB/s

A GH200 is a CPU and a GPU on one module



- The GPU computes. The CPU reads the data, runs Python, and keeps it fed
- HBM3 is memory stacked on the GPU package: only 96 GB, but 4 TB/s
- Between them 900 GB/s, about seven times what a plug-in card gets

☰ One operation, and who does it

The atelier against Alps

A collection of computers	Rate	Power
Prony's atelier, 1795 twenty people	0.694 additions/s	about 2 kW

The atelier against Alps

A collection of computers	Rate	Power
Prony's atelier, 1795 twenty people	0.694 additions/s	about 2 kW
Alps, 2024 10,752 modules	10.64 EFLOP/s	7.1 MW

The atelier against Alps

A collection of computers	Rate	Power
Prony's atelier, 1795 twenty people	0.694 additions/s	about 2 kW
Alps, 2024 10,752 modules	10.64 EFLOP/s	7.1 MW

Fifteen billion billion times the rate, on three and a half thousand times the power

☰ One operation, and who does it

One second of a GH200, in three registers

The same one second of arithmetic

The mean age of all 8.2 billion people alive,
one addition each

8.3 microseconds

One second of a GH200, in three registers

The same one second of arithmetic

The mean age of all 8.2 billion people alive,
one addition each

8.3 microseconds

130,000 photographs classified by ResNet-50,
each 224 by 224 pixels, or 0.05 megapixels

one second

One second of a GH200, in three registers

The same one second of arithmetic

The mean age of all 8.2 billion people alive,
one addition each

8.3 microseconds

130,000 photographs classified by ResNet-50,
each 224 by 224 pixels, or 0.05 megapixels

one second

Prony's atelier doing that same
one second of work

45 to 136 million years

One second of a GH200, in three registers

The same one second of arithmetic

The mean age of all 8.2 billion people alive,
one addition each

8.3 microseconds

130,000 photographs classified by ResNet-50,
each 224 by 224 pixels, or 0.05 megapixels

one second

Prony's atelier doing that same
one second of work

45 to 136 million years

- What changed is not the idea. It is the price of one operation

One second of a GH200, in three registers

The same one second of arithmetic

The mean age of all 8.2 billion people alive,
one addition each

8.3 microseconds

130,000 photographs classified by ResNet-50,
each 224 by 224 pixels, or 0.05 megapixels

one second

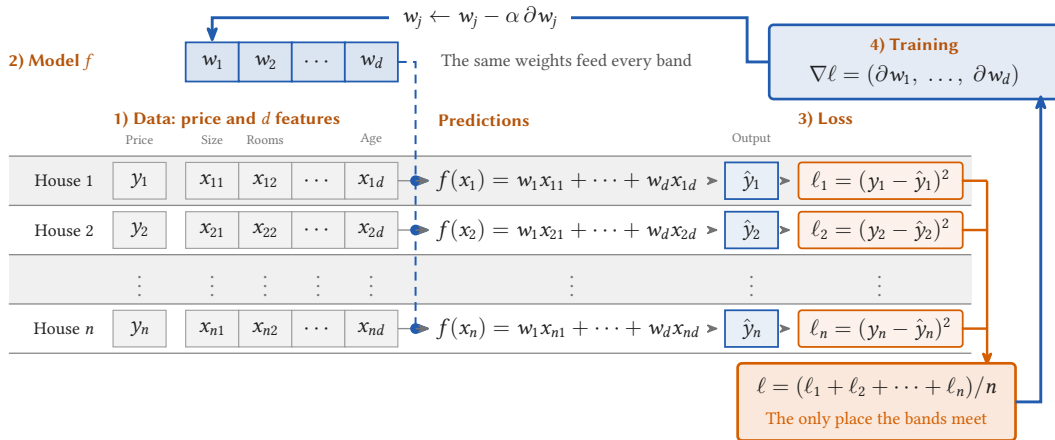
Prony's atelier doing that same
one second of work

45 to 136 million years

- What changed is not the idea. It is the price of one operation

ResNet-50: He et al., [arXiv:1512.03385](https://arxiv.org/abs/1512.03385), 3.8 billion multiply-adds, so 7.6 GFLOP per image. A peak count.

One training step of linear regression





Section 3

What the headline rate hides

3

One floating point number, taken apart

Start with

$$\pi = 3.14159265 \dots$$

One floating point number, taken apart

Start with

$$\pi = 3.14159265 \dots$$

A mantissa times a power of two

$$1.5707963 \dots \times 2^1$$

One floating point number, taken apart

Start with

$$\pi = 3.14159265 \dots$$

A mantissa times a power of two

$$1.5707963 \dots \times 2^1$$

So really two numbers

mantissa 1.5707963 ...
exponent 1

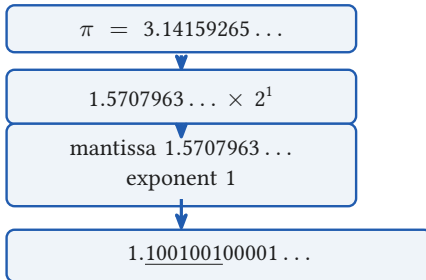
One floating point number, taken apart

Start with

A mantissa times a power of two

So really two numbers

The mantissa in binary



One floating point number, taken apart

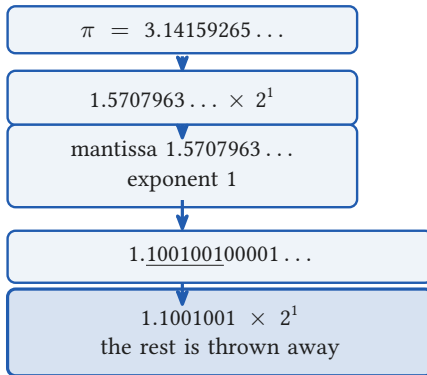
Start with

A mantissa times a power of two

So really two numbers

The mantissa in binary

bf16 keeps seven of those bits



What the seven bits cost

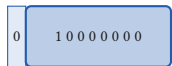
0

sign

The sixteen bits

What the seven bits cost

The sixteen bits

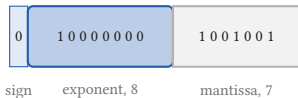


sign

exponent, 8

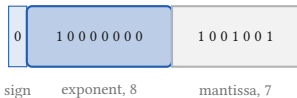
What the seven bits cost

The sixteen bits



What the seven bits cost

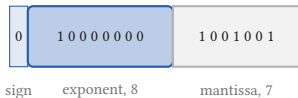
The sixteen bits



Back to decimal
3.140625

What the seven bits cost

The sixteen bits

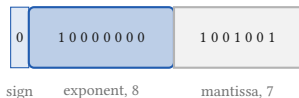


Back to decimal
3.140625

We started from
3.14159265...

What the seven bits cost

The sixteen bits



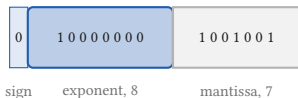
Back to decimal
3.140625

We started from
3.14159265...

Lost 0.00097, or 0.03 percent

What the seven bits cost

The sixteen bits



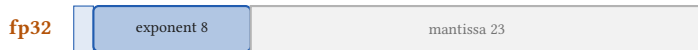
Back to decimal
3.140625

We started from
3.14159265...

Lost 0.00097, or 0.03 percent

- Seven bits cannot hold π , so the number stored is not the number you asked for

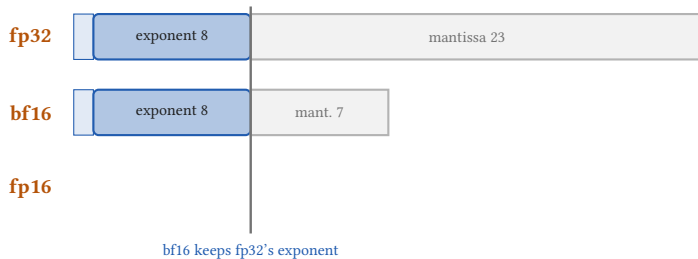
Not every FLOP costs the same



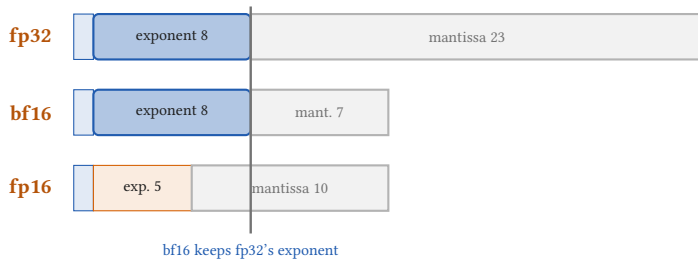
bf16

fp16

Not every FLOP costs the same



Not every FLOP costs the same



☰ What the headline rate hides

Why we say dense BF16, and not just BF16

The datasheet's fast number assumes a pattern

Why we say dense BF16, and not just BF16

The datasheet's fast number assumes a pattern



Why we say dense BF16, and not just BF16

The datasheet's fast number assumes a pattern



Two of every four pinned to zero, so the hardware skips them
1,979 TFLOP/s

Why we say dense BF16, and not just BF16

The datasheet's fast number assumes a pattern



Two of every four pinned to zero, so the hardware skips them

1,979 TFLOP/s

Pretraining cannot promise it

Why we say dense BF16, and not just BF16

The datasheet's fast number assumes a pattern



Two of every four pinned to zero, so the hardware skips them

1,979 TFLOP/s

Pretraining cannot promise it



Why we say dense BF16, and not just BF16

The datasheet's fast number assumes a pattern



Two of every four pinned to zero, so the hardware skips them

1,979 TFLOP/s

Pretraining cannot promise it



Every weight starts random and has to stay free to move

990 TFLOP/s per GH200, dense

Why we say dense BF16, and not just BF16

The datasheet's fast number assumes a pattern



Two of every four pinned to zero, so the hardware skips them
1,979 TFLOP/s

Pretraining cannot promise it



Every weight starts random and has to stay free to move
990 TFLOP/s per GH200, dense

- Sparsity is just something you impose on a finished model

Capacity and bandwidth are different numbers

96 GB
how much fits

Capacity and bandwidth are different numbers

96 GB
how much fits

4 TB/s
how fast you can move it

Capacity and bandwidth are different numbers

96 GB
how much fits

4 TB/s
how fast you can move it

Decides whether the weights and the optimiser state fit on one device at all

Capacity and bandwidth are different numbers

96 GB
how much fits

Decides whether the weights and the optimiser state fit on one device at all

4 TB/s
how fast you can move it

Decides how quickly the arithmetic units can be fed once the data is there

Capacity and bandwidth are different numbers

96 GB
how much fits

Decides whether the weights and the optimiser state fit on one device at all

4 TB/s
how fast you can move it

Decides how quickly the arithmetic units can be fed once the data is there

Neither substitutes for the other

Capacity and bandwidth are different numbers

96 GB
how much fits

Decides whether the weights and the optimiser state fit on one device at all

4 TB/s
how fast you can move it

Decides how quickly the arithmetic units can be fed once the data is there

Neither substitutes for the other

- 96 GB is less than an ordinary server holds. That is the trade: HBM is stacked on the package and wired short and wide, which is what buys the speed

Two hundred and forty six operations per byte

HBM3
4 TB/s

Two hundred and forty six operations per byte



What the headline rate hides

Two hundred and forty six operations per byte



Two hundred and forty six operations per byte



$$\frac{990 \times 10^{12} \text{ FLOP/s}}{4.023 \times 10^{12} \text{ bytes/s}} = 246 \text{ FLOP per byte}$$

Two hundred and forty six operations per byte



$$\frac{990 \times 10^{12} \text{ FLOP/s}}{4.023 \times 10^{12} \text{ bytes/s}} = 246 \text{ FLOP per byte}$$

A rate over a rate: the seconds cancel, and what is left is operations per byte

Two hundred and forty six operations per byte



$$\frac{990 \times 10^{12} \text{ FLOP/s}}{4.023 \times 10^{12} \text{ bytes/s}} = 246 \text{ FLOP per byte}$$

A rate over a rate: the seconds cancel, and what is left is operations per byte

So unless the computation reuses each byte about that many times, the units wait. A large matrix multiply does reuse. Many other operations do not.

Two hundred and forty six operations per byte



$$\frac{990 \times 10^{12} \text{ FLOP/s}}{4.023 \times 10^{12} \text{ bytes/s}} = 246 \text{ FLOP per byte}$$

A rate over a rate: the seconds cancel, and what is left is operations per byte

So unless the computation reuses each byte about that many times, the units wait. A large matrix multiply does reuse. Many other operations do not.

- This is one of the reasons a real run reaches 21 to 46 percent of peak



Section 4

The machine, and what it draws

4

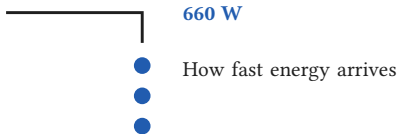
A watt is a rate, a kilowatt hour is an amount



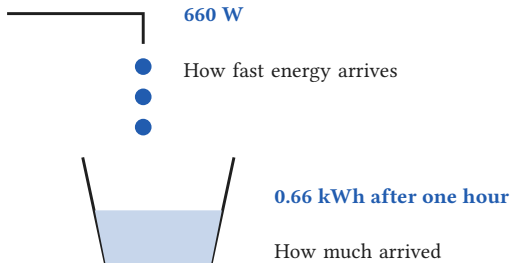
660 W

How fast energy arrives

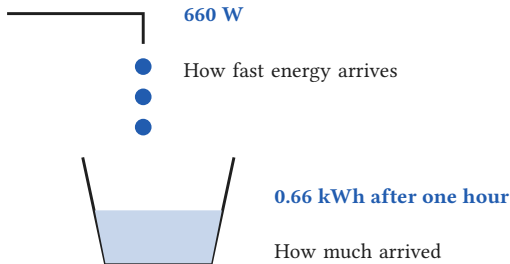
A watt is a rate, a kilowatt hour is an amount



A watt is a rate, a kilowatt hour is an amount



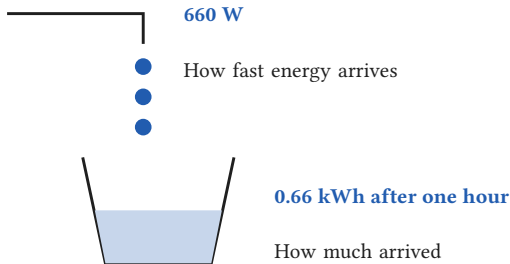
A watt is a rate, a kilowatt hour is an amount



Watch the slash

km/h the slash divides,
so it is a speed

A watt is a rate, a kilowatt hour is an amount

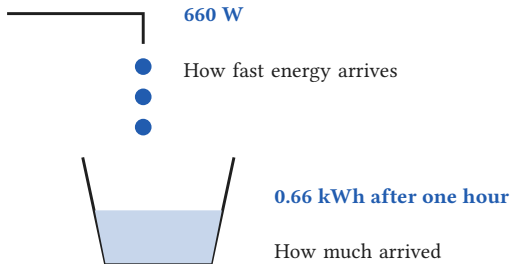


Watch the slash

km/h the slash divides,
so it is a speed

kWh no slash. The hour
multiplies, so it is an amount

A watt is a rate, a kilowatt hour is an amount



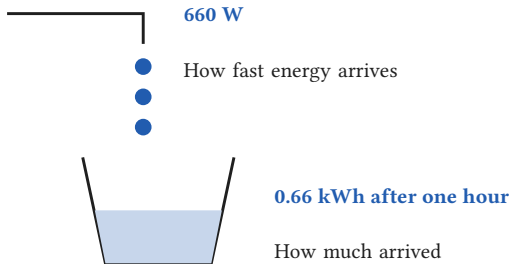
Watch the slash

km/h the slash divides,
so it is a speed

kWh no slash. The hour
multiplies, so it is an amount

Longer gives you *more*, so the hour
cannot be dividing

A watt is a rate, a kilowatt hour is an amount



Watch the slash

km/h the slash divides,
so it is a speed

kWh no slash. The hour
multiplies, so it is an amount

Longer gives you *more*, so the hour
cannot be dividing

- Rate $\times$ time = amount, twice over: FLOP/s $\times$ seconds = FLOPs, and watts $\times$ hours = kWh

A wattage becomes a bill in two multiplications

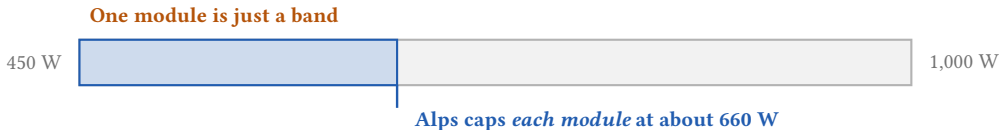
One module is just a band

450 W

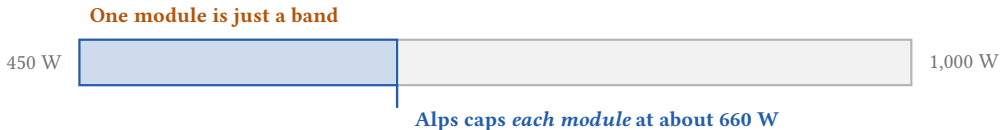


1,000 W

A wattage becomes a bill in two multiplications

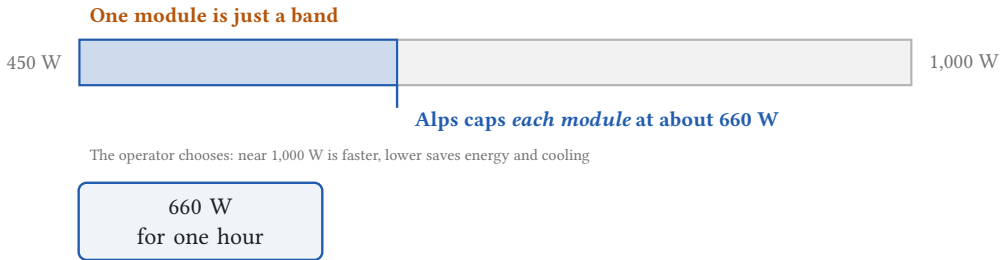


A wattage becomes a bill in two multiplications

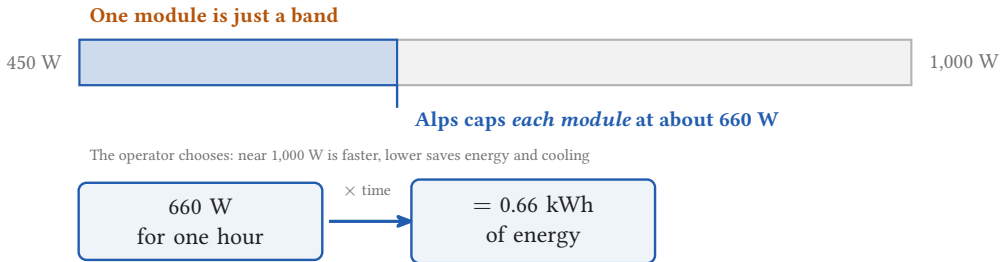


The operator chooses: near 1,000 W is faster, lower saves energy and cooling

A wattage becomes a bill in two multiplications

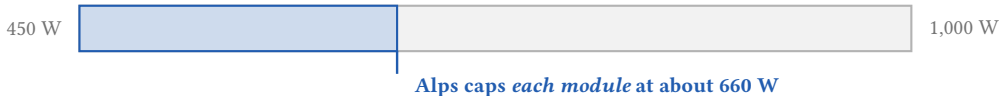


A wattage becomes a bill in two multiplications

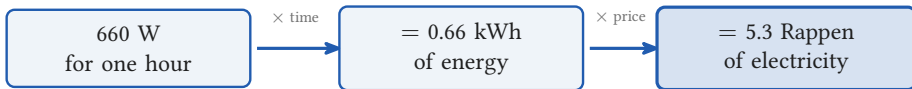


A wattage becomes a bill in two multiplications

One module is just a band



The operator chooses: near 1,000 W is faster, lower saves energy and cooling

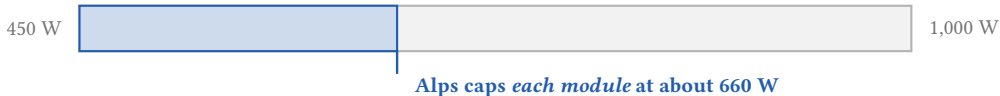


450 to 1,000 W: GH200 datasheet. 660 W: the Alps cap, docs.cscs.ch/running/slurm. 8 Rappen per kWh assumed (ElCom 2026 large consumer band 6.75 to 9; households pay 27.7).

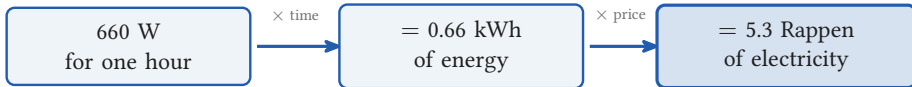
Check: $10,752 \times 660 \text{ W}$ is 7,096 kW against the 7,124 kW TOP500 measured during HPL, top500.org/system/180259.

A wattage becomes a bill in two multiplications

One module is just a band



The operator chooses: near 1,000 W is faster, lower saves energy and cooling

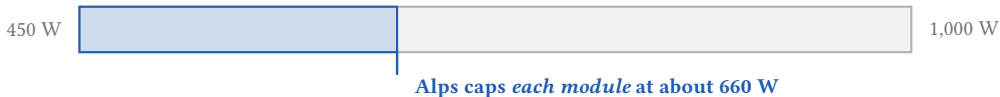


450 to 1,000 W: GH200 datasheet. 660 W: the Alps cap, docs.cscs.ch/running/slurm. 8 Rappen per kWh assumed (ElCom 2026 large consumer band 6.75 to 9; households pay 27.7).
Check: $10,752 \times 660 \text{ W}$ is 7,096 kW against the 7,124 kW TOP500 measured during HPL, top500.org/system/180259.

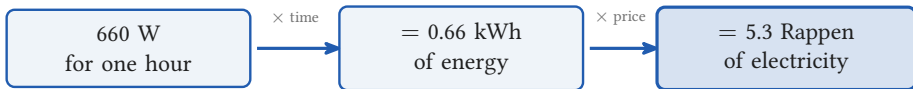
- That is **one module**. All 10,752 is 7.1 megawatts, about 570 francs an hour

A wattage becomes a bill in two multiplications

One module is just a band



The operator chooses: near 1,000 W is faster, lower saves energy and cooling



450 to 1,000 W: GH200 datasheet. 660 W: the Alps cap, docs.cscs.ch/running/slurm. 8 Rappen per kWh assumed (ElCom 2026 large consumer band 6.75 to 9; households pay 27.7).
Check: $10,752 \times 660 \text{ W}$ is 7,096 kW against the 7,124 kW TOP500 measured during HPL, top500.org/system/180259.

- That is **one module**. All 10,752 is 7.1 megawatts, about 570 francs an hour
- The whole Apertus run is 3.96 GWh, about 0.3 million francs

Alps is an HPE Cray EX254n

The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*

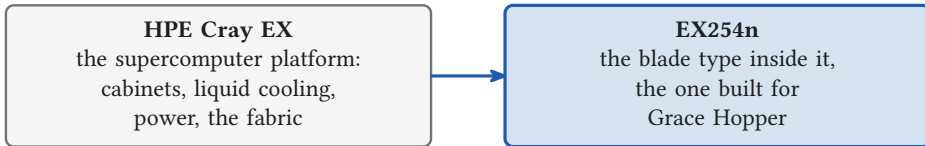
Alps is an HPE Cray EX254n

The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*

HPE Cray EX
the supercomputer platform:
cabinets, liquid cooling,
power, the fabric

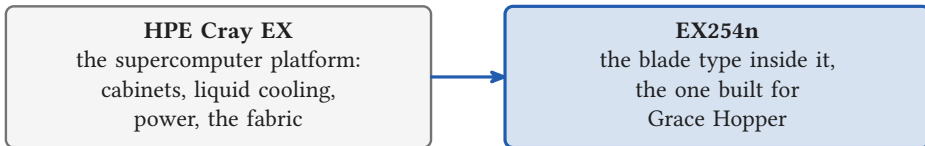
Alps is an HPE Cray EX254n

The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*



Alps is an HPE Cray EX254n

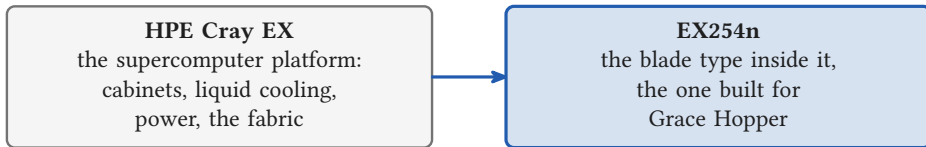
The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*



The trailing n marks the NVIDIA accelerated blades

Alps is an HPE Cray EX254n

The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*



The trailing n marks the NVIDIA accelerated blades

- The blade choice is what makes Alps a machine for AI rather than a conventional CPU cluster

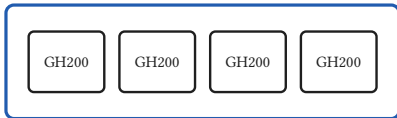
A node is not a socket

One node



A node is not a socket

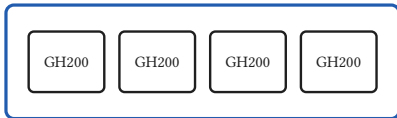
One node



One operating system, one address space

A node is not a socket

One node



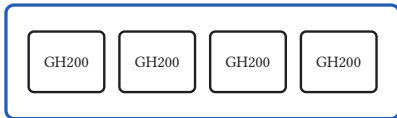
One operating system, one address space

Socket one processor module

Node one independent computer

A node is not a socket

One node



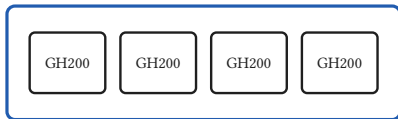
Socket one processor module
Node one independent computer

One operating system, one address space

$$2,688 \text{ nodes} \times 4 \text{ modules} = 10,752 \text{ sockets}$$

A node is not a socket

One node



Socket one processor module
Node one independent computer

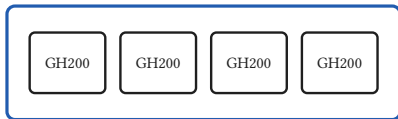
One operating system, one address space

$2,688 \text{ nodes} \times 4 \text{ modules} = 10,752 \text{ sockets}$

Socket to socket, inside the node: 150 GB/s

A node is not a socket

One node



Socket one processor module

Node one independent computer

One operating system, one address space

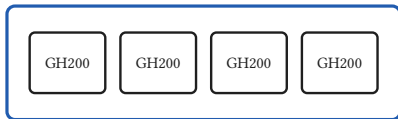
$2,688 \text{ nodes} \times 4 \text{ modules} = 10,752 \text{ sockets}$

Socket to socket, inside the node: 150 GB/s

Between nodes: 25 GB/s per GPU, six times less

A node is not a socket

One node



Socket one processor module

Node one independent computer

One operating system, one address space

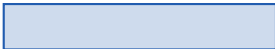
$2,688 \text{ nodes} \times 4 \text{ modules} = 10,752 \text{ sockets}$

Socket to socket, inside the node: 150 GB/s

Between nodes: 25 GB/s per GPU, six times less

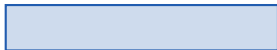
- The node boundary is where the bandwidth falls off

Four links, and each step down costs you

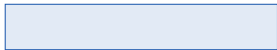


4,023 GB/s the GPU to its own HBM3

Four links, and each step down costs you

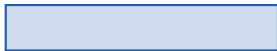


4,023 GB/s the GPU to its own HBM3

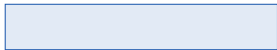


900 GB/s Grace to Hopper, inside one module

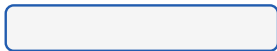
Four links, and each step down costs you



4,023 GB/s the GPU to its own HBM3

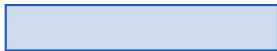


900 GB/s Grace to Hopper, inside one module

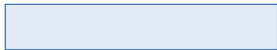


150 GB/s socket to socket, inside a node

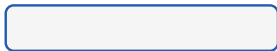
Four links, and each step down costs you



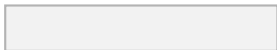
4,023 GB/s the GPU to its own HBM3



900 GB/s Grace to Hopper, inside one module

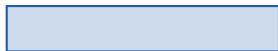


150 GB/s socket to socket, inside a node

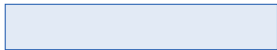


25 GB/s per GPU between nodes

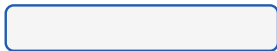
Four links, and each step down costs you



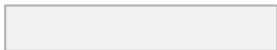
4,023 GB/s the GPU to its own HBM3



900 GB/s Grace to Hopper, inside one module



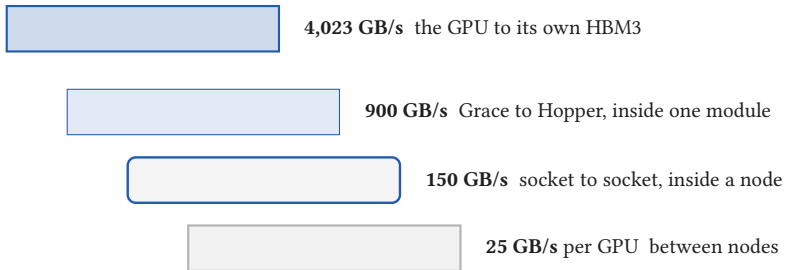
150 GB/s socket to socket, inside a node



25 GB/s per GPU between nodes

Each step down costs about a factor of five. Top to bottom, 161 times.

Four links, and each step down costs you



Each step down costs about a factor of five. Top to bottom, 161 times.

- So you keep the data at the fast end and cross the slow links as rarely as you can

What the TOP500 actually measures

A list of the 500 most powerful known supercomputers, published every June and November since 1993.
Alps is rank 10 in June 2026.

What the TOP500 actually measures

A list of the 500 most powerful known supercomputers, published every June and November since 1993. Alps is rank 10 in June 2026.

It ranks them on one benchmark: **HPL**, solving a large dense linear system in **64 bit** arithmetic

What the TOP500 actually measures

A list of the 500 most powerful known supercomputers, published every June and November since 1993. Alps is rank 10 in June 2026.

It ranks them on one benchmark: **HPL**, solving a large dense linear system in **64 bit** arithmetic

Rmax 434.90 PFLOP/s
what it actually achieved

What the TOP500 actually measures

A list of the 500 most powerful known supercomputers, published every June and November since 1993.
Alps is rank 10 in June 2026.

It ranks them on one benchmark: **HPL**, solving a large dense linear system in **64 bit** arithmetic

Rmax 434.90 PFLOP/s
what it actually achieved

Rpeak 574.84 PFLOP/s
the theoretical maximum

What the TOP500 actually measures

A list of the 500 most powerful known supercomputers, published every June and November since 1993. Alps is rank 10 in June 2026.

It ranks them on one benchmark: **HPL**, solving a large dense linear system in **64 bit** arithmetic

Rmax 434.90 PFLOP/s
what it actually achieved

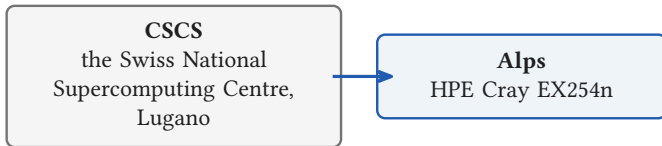
Rpeak 574.84 PFLOP/s
the theoretical maximum

So even on its own benchmark it reaches 75.7 percent, and drew 7.1 MW doing it

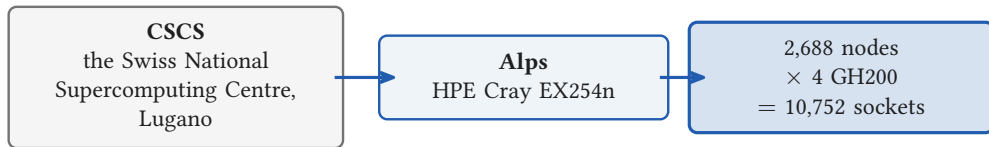
CSCS and Alps, the machine behind the anchor

CSCS
the Swiss National
Supercomputing Centre,
Lugano

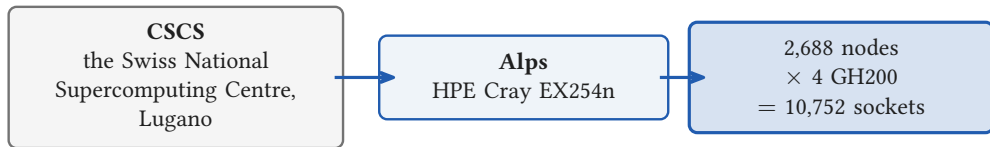
CSCS and Alps, the machine behind the anchor



CSCS and Alps, the machine behind the anchor

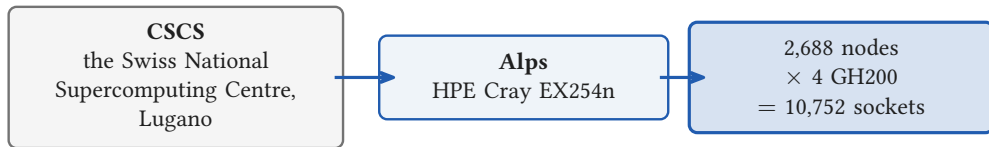


CSCS and Alps, the machine behind the anchor



At dense BF16 the whole partition is about **10.6 EFLOP/s**, which is the rate that matters for training.

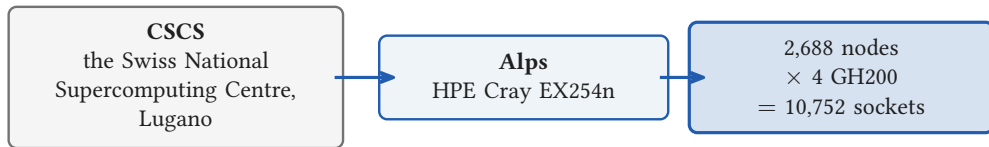
CSCS and Alps, the machine behind the anchor



At dense BF16 the whole partition is about **10.6 EFLOP/s**, which is the rate that matters for training.

Its TOP500 entry, 434.90 PFLOP/s at 7,124 kW, is an **fp64** measurement and is about 24 times smaller.

CSCS and Alps, the machine behind the anchor



At dense BF16 the whole partition is about **10.6 EFLOP/s**, which is the rate that matters for training.

Its TOP500 entry, 434.90 PFLOP/s at 7,124 kW, is an **fp64** measurement and is about 24 times smaller.

Machine: top500.org/system/180259 and cscs.ch/computers/alps.



Section 5

From a model to GPU hours

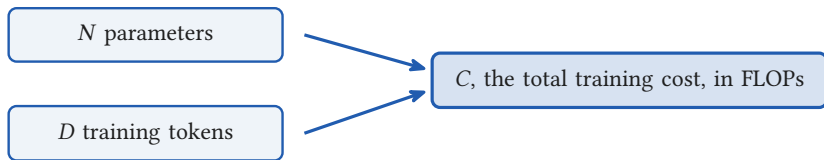
5

Two numbers price a training run

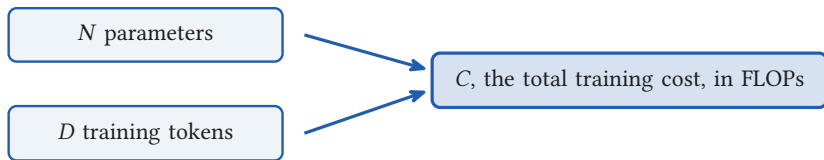
N parameters

D training tokens

Two numbers price a training run

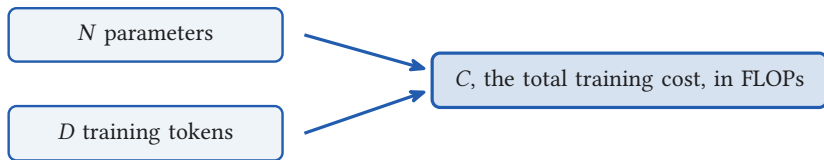


Two numbers price a training run



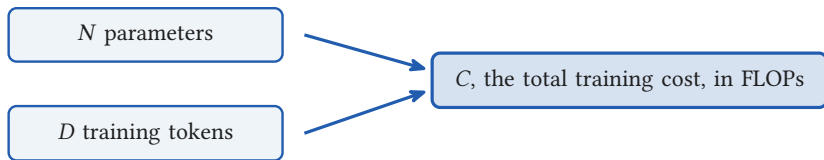
$$C = \underbrace{N}_{\text{parameters}}$$

Two numbers price a training run



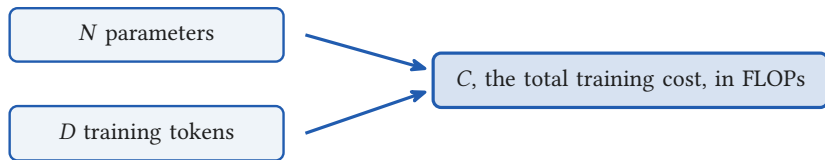
$$C = \underbrace{N}_{\text{parameters}} \times \underbrace{D}_{\text{tokens}}$$

Two numbers price a training run



$$C = \underbrace{6}_{\text{per parameter, per token}} \times \underbrace{N}_{\text{parameters}} \times \underbrace{D}_{\text{tokens}}$$

Two numbers price a training run



$$C = \underbrace{6}_{\text{per parameter, per token}} \times \underbrace{N}_{\text{parameters}} \times \underbrace{D}_{\text{tokens}}$$

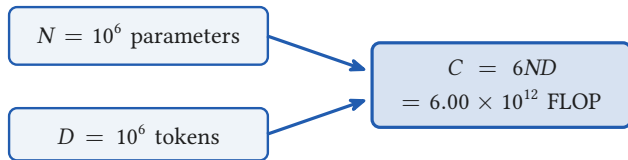
[Kaplan et al. 2020](#), section 2.1. Where the 6 comes from is the 12:00 lecture; for the next fifty minutes it is a rule of thumb.

A million by a million is two minutes

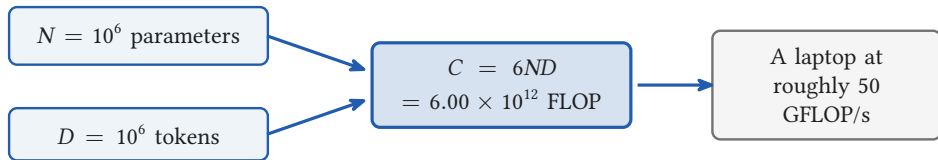
$$N = 10^6 \text{ parameters}$$

$$D = 10^6 \text{ tokens}$$

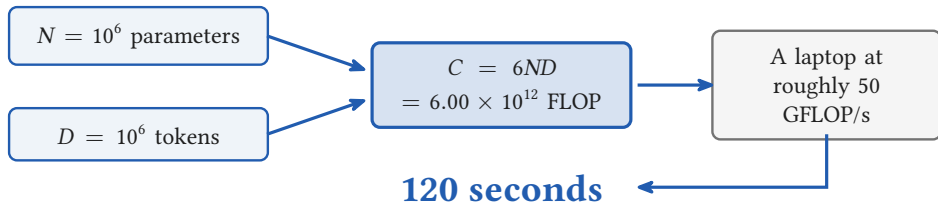
A million by a million is two minutes



A million by a million is two minutes



A million by a million is two minutes



The anchor: Apertus, trained on Alps

Apertus-70B

Released 2 September 2025, Apache 2.0

70.60×10^9 parameters, 15 trillion tokens

1,811 languages, 40 percent of it not English

The anchor: Apertus, trained on Alps

Apertus-70B

Released 2 September 2025, Apache 2.0
 70.60×10^9 parameters, 15 trillion tokens
1,811 languages, 40 percent of it not English

Alps, at CSCS Lugano

10,752 GH200 sockets, 4 per node
990 TFLOP/s dense BF16 each
TOP500: 434.90 PFlop/s at 7,124 kW

The anchor: Apertus, trained on Alps

Apertus-70B

Released 2 September 2025, Apache 2.0
 70.60×10^9 parameters, 15 trillion tokens
1,811 languages, 40 percent of it not English

Alps, at CSCS Lugano

10,752 GH200 sockets, 4 per node
990 TFLOP/s dense BF16 each
TOP500: 434.90 PFlop/s at 7,124 kW

The production run Over 6 million GPU hours, over roughly three months, on up to 4,096 GPUs, sustaining about 723 tokens per second per GPU

The anchor: Apertus, trained on Alps

Apertus-70B

Released 2 September 2025, Apache 2.0
 70.60×10^9 parameters, 15 trillion tokens
1,811 languages, 40 percent of it not English

Alps, at CSCS Lugano

10,752 GH200 sockets, 4 per node
990 TFLOP/s dense BF16 each
TOP500: 434.90 PFlop/s at 7,124 kW

The production run Over 6 million GPU hours, over roughly three months, on up to 4,096 GPUs, sustaining about 723 tokens per second per GPU

Built by EPFL, ETH Zurich and CSCS. Every number in the next forty minutes is about this run.

The anchor: Apertus, trained on Alps

Apertus-70B

Released 2 September 2025, Apache 2.0
 70.60×10^9 parameters, 15 trillion tokens
1,811 languages, 40 percent of it not English

Alps, at CSCS Lugano

10,752 GH200 sockets, 4 per node
990 TFLOP/s dense BF16 each
TOP500: 434.90 PFlop/s at 7,124 kW

The production run Over 6 million GPU hours, over roughly three months, on up to 4,096 GPUs, sustaining about 723 tokens per second per GPU

Built by EPFL, ETH Zurich and CSCS. Every number in the next forty minutes is about this run.

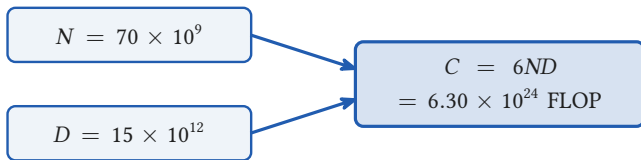
Model: [arXiv:2509.14233](https://arxiv.org/abs/2509.14233). Run: [arXiv:2604.12973](https://arxiv.org/abs/2604.12973). Machine: top500.org/system/180259.

The same formula on Apertus-70B

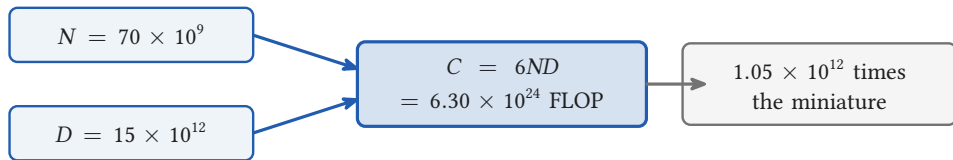
$$N = 70 \times 10^9$$

$$D = 15 \times 10^{12}$$

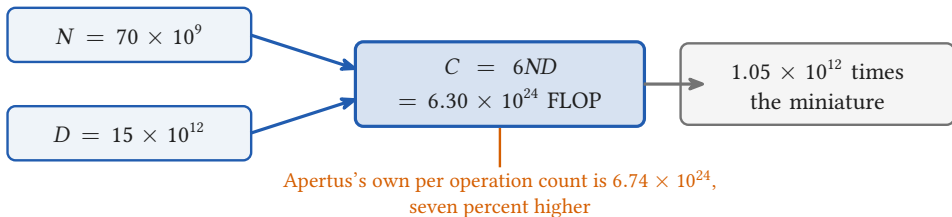
The same formula on Apertus-70B



The same formula on Apertus-70B



The same formula on Apertus-70B



From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion tokens, on up to 4,096 GPUs.

From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion tokens, on up to 4,096 GPUs.

4,096 GPUs flat out
for **61 days**

From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion tokens, on up to 4,096 GPUs.

4,096 GPUs flat out
for **61 days**

It spanned about 91 days,
so **2,747** on average

From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion tokens, on up to 4,096 GPUs.

4,096 GPUs flat out
for **61 days**

It spanned about 91 days,
so **2,747** on average

The gap is ramp up, restarts and smaller phases: a campaign is not one uninterrupted run at full width

From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion tokens, on up to 4,096 GPUs.

4,096 GPUs flat out
for **61 days**

It spanned about 91 days,
so **2,747** on average

The gap is ramp up, restarts and smaller phases: a campaign is not one uninterrupted run at full width

$6 \text{ million GPU hours} \times 660 \text{ W} = \mathbf{3.96 \text{ GWh}} \times 8 \text{ Rappen} = \mathbf{0.32 \text{ million francs}}$



Section 6

Peak, and what a real run gets

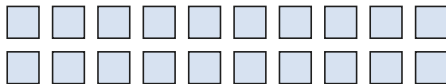
6

Counting the workshop is not measuring it



Twenty computers, one desk each

Counting the workshop is not measuring it



Twenty computers, one desk each

Count the desks,
multiply by how fast
one clerk works

Counting the workshop is not measuring it



Twenty computers, one desk each

Count the desks,
multiply by how fast
one clerk works



That is the workshop's
peak

Counting the workshop is not measuring it



Twenty computers, one desk each

Counted from the doorstep. Nothing was run.

Count the desks,
multiply by how fast
one clerk works



That is the workshop's
peak

Counting the workshop is not measuring it



Twenty computers, one desk each

Counted from the doorstep. Nothing was run.

Now watch for an hour

Count the desks,
multiply by how fast
one clerk works



That is the workshop's
peak

Counting the workshop is not measuring it



Twenty computers, one desk each

Counted from the doorstep. Nothing was run.

Now watch for an hour

A clerk waits for the next sheet.

Two clerks stop to compare a figure.

The batch is small, so some desks are empty.

Count the desks,
multiply by how fast
one clerk works



That is the workshop's
peak

Counting the workshop is not measuring it



Twenty computers, one desk each

Counted from the doorstep. Nothing was run.

Now watch for an hour

A clerk waits for the next sheet.

Two clerks stop to compare a figure.

The batch is small, so some desks are empty.

Count the desks,
multiply by how fast
one clerk works



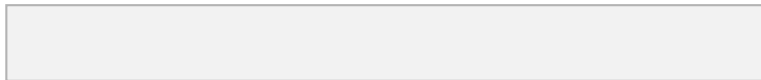
That is the workshop's
peak

- The clerks were at full speed the whole time. They were not always given something to do

☰ Peak, and what a real run gets

Peak is just a ceiling

One GH200, one second

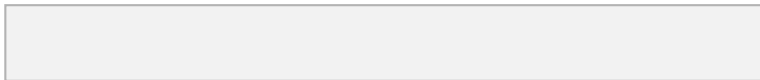


Peak

☰ Peak, and what a real run gets

Peak is just a ceiling

One GH200, one second



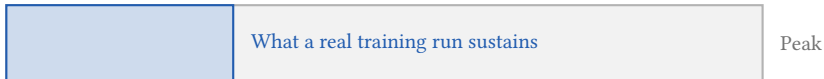
Peak

every arithmetic unit busy, every cycle, data always ready

☰ Peak, and what a real run gets

Peak is just a ceiling

One GH200, one second

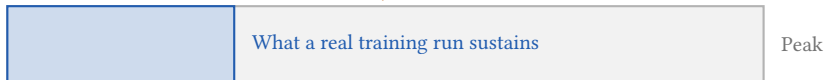


every arithmetic unit busy, every cycle, data always ready

A whole training run, averaged, drawn on one second so you can see it

Peak is just a ceiling

One GH200, one second

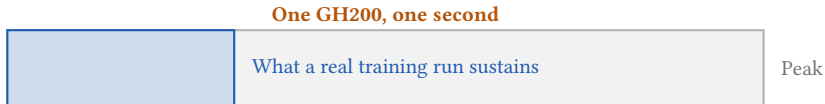


every arithmetic unit busy, every cycle, data always ready

A whole training run, averaged, drawn on one second so you can see it

The gap is waiting on memory, talking to other chips, and kernels that do not fill the hardware.

Peak is just a ceiling



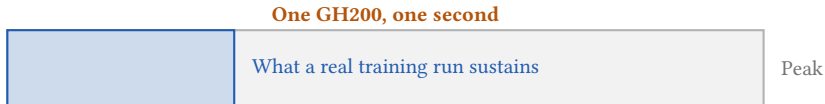
every arithmetic unit busy, every cycle, data always ready

A whole training run, averaged, drawn on one second so you can see it

The gap is waiting on memory, talking to other chips, and kernels that do not fill the hardware.

- Nobody reaches peak. The datasheet number is a ceiling

Peak is just a ceiling



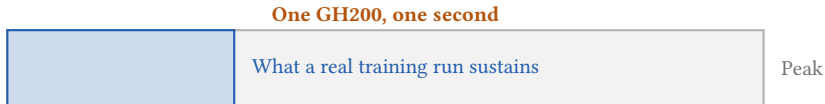
every arithmetic unit busy, every cycle, data always ready

A whole training run, averaged, drawn on one second so you can see it

The gap is waiting on memory, talking to other chips, and kernels that do not fill the hardware.

- Nobody reaches peak. The datasheet number is a ceiling
- The share you do reach is [model FLOPs utilisation](#), written MFU

Peak is just a ceiling



every arithmetic unit busy, every cycle, data always ready

A whole training run, averaged, drawn on one second so you can see it

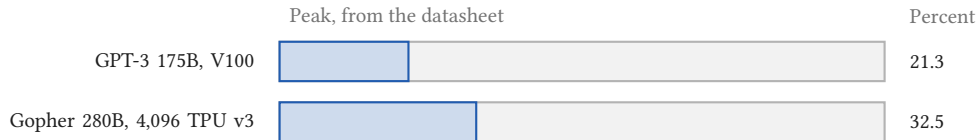
The gap is waiting on memory, talking to other chips, and kernels that do not fill the hardware.

- Nobody reaches peak. The datasheet number is a ceiling
- The share you do reach is [model FLOPs utilisation](#), written MFU
- It is the blue bar over the whole bar, so MFU 0.30 means thirty percent of peak

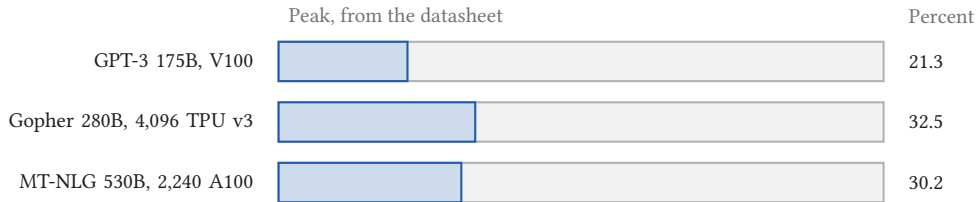
Published runs reach 21 to 46 percent of peak



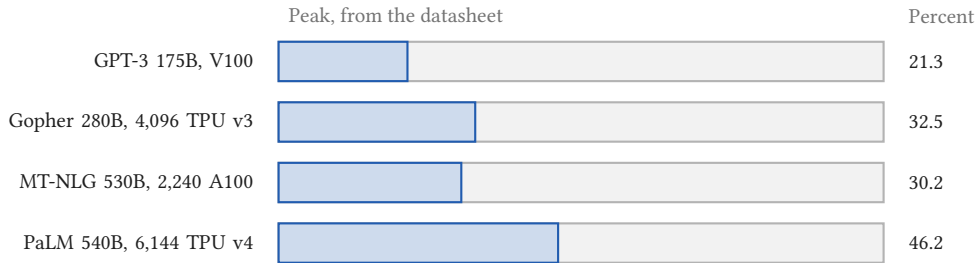
Published runs reach 21 to 46 percent of peak



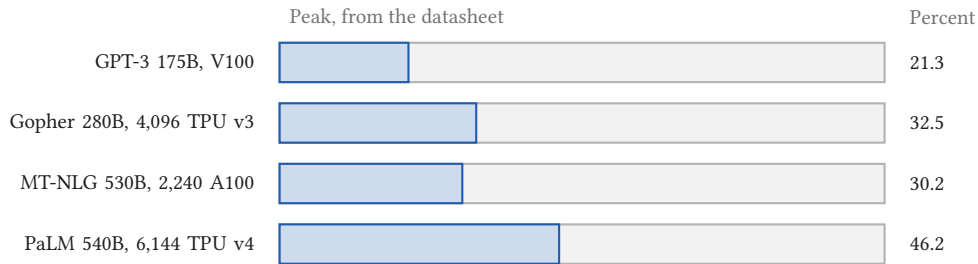
Published runs reach 21 to 46 percent of peak



Published runs reach 21 to 46 percent of peak

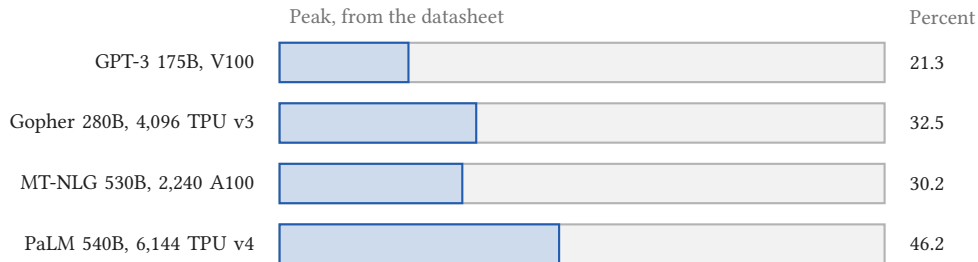


Published runs reach 21 to 46 percent of peak



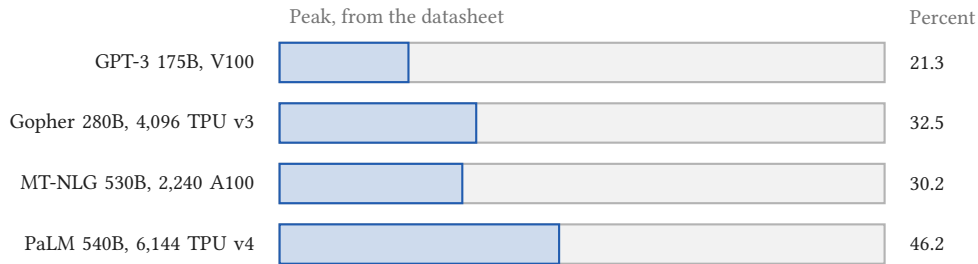
- Two ran on TPUs, so the umbrella word is [accelerator](#): GPU, TPU, or GH200

Published runs reach 21 to 46 percent of peak



- Two ran on TPUs, so the umbrella word is [accelerator](#): GPU, TPU, or GH200
- Model FLOPs utilisation: the share of peak a training run reaches

Published runs reach 21 to 46 percent of peak



- Two ran on TPUs, so the umbrella word is [accelerator](#): GPU, TPU, or GH200
- Model FLOPs utilisation: the share of peak a training run reaches
- Assuming 100 percent is wrong by a factor of two to five

Where the utilisation formula comes from

$$\text{MFU} = \frac{\text{observed throughput}}{\text{theoretical maximum}}$$

Where the utilisation formula comes from

$$\text{MFU} = \frac{\text{tokens/s} \times \text{FLOP per token}}{\text{peak FLOP/s} \times \text{chips}}$$

- Tokens per second is what you observe. FLOP per token is what you count

Where the utilisation formula comes from

$$\text{MFU} = \frac{\frac{D}{T} \times 6N}{\text{peak FLOP/s} \times \text{chips}}$$

- Tokens per second is what you observe. FLOP per token is what you count
- Substitute D/T tokens per second, and $6N$ FLOP per token, where T is the wall clock duration of the training run

Where the utilisation formula comes from

$$\text{MFU} = \frac{6ND}{T \times \text{chips} \times \text{peak FLOP/s}}$$

- Tokens per second is what you observe. FLOP per token is what you count
- Substitute D/T tokens per second, and $6N$ FLOP per token, where T is the wall clock duration of the training run

Where the utilisation formula comes from

$$\text{MFU} = \frac{6ND}{\underbrace{T \times \text{chips}}_{\text{GPU-seconds}} \times \text{peak FLOP/s}}$$

- Tokens per second is what you observe. FLOP per token is what you count
- Substitute D/T tokens per second, and $6N$ FLOP per token, where T is the wall clock duration of the training run
- T times chips is exactly GPU-seconds, which is what the run reports

Where the utilisation formula comes from

$$\text{MFU} = \frac{6ND}{\text{GPU-hours} \times 3,600 \text{ s/h} \times \text{peak FLOP/s}}$$

- Tokens per second is what you observe. FLOP per token is what you count
- Substitute D/T tokens per second, and $6N$ FLOP per token, where T is the wall clock duration of the training run
- T times chips is exactly GPU-seconds, which is what the run reports

Where the utilisation formula comes from

$$\text{MFU} = \frac{6ND}{\text{GPU-hours} \times 3,600 \text{ s/h} \times \text{peak FLOP/s}}$$

- Tokens per second is what you observe. FLOP per token is what you count
- Substitute D/T tokens per second, and $6N$ FLOP per token, where T is the wall clock duration of the training run
- T times chips is exactly GPU-seconds, which is what the run reports
- So the measurement enters through the hours, and $6ND$ supplies the work

Six million hours, measured against peak

The consumption

Over 6 million GPU hours, the 70B production run

Six million hours, measured against peak

The consumption

Over 6 million GPU hours, the 70B production run

$$\begin{aligned} &6 \times 10^6 \text{ GPU-h} \times 3,600 \text{ s/h} \times 990 \text{ TFLOP/s per GPU} \\ &= 2.14 \times 10^{25} \text{ FLOP of peak capacity} \end{aligned}$$

Six million hours, measured against peak

The consumption

Over 6 million GPU hours, the 70B production run

measured
× datasheet

$$\begin{aligned} &6 \times 10^6 \text{ GPU-h} \times 3,600 \text{ s/h} \times 990 \text{ TFLOP/s per GPU} \\ &= 2.14 \times 10^{25} \text{ FLOP of peak capacity} \end{aligned}$$

Six million hours, measured against peak

The consumption

Over 6 million GPU hours, the 70B production run

measured
× datasheet

$$6 \times 10^6 \text{ GPU-h} \times 3,600 \text{ s/h} \times 990 \text{ TFLOP/s per GPU} \\ = 2.14 \times 10^{25} \text{ FLOP of peak capacity}$$

analytic

$$\frac{6.30 \times 10^{24}}{2.14 \times 10^{25}} = 29.5 \text{ percent}$$

6ND for the 70B



Section 7

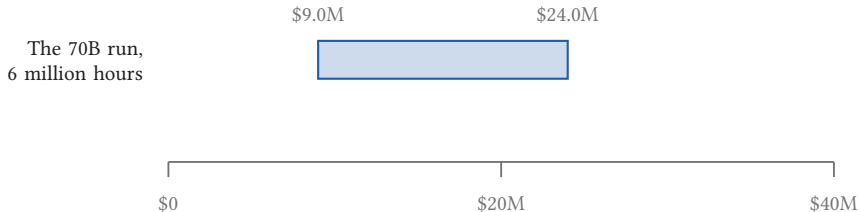
Dollars and energy

7

Six million hours, priced at cloud rates

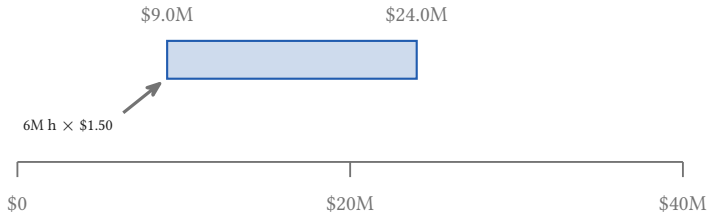


Six million hours, priced at cloud rates



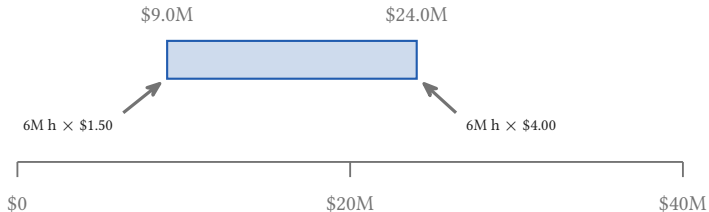
Six million hours, priced at cloud rates

The 70B run,
6 million hours



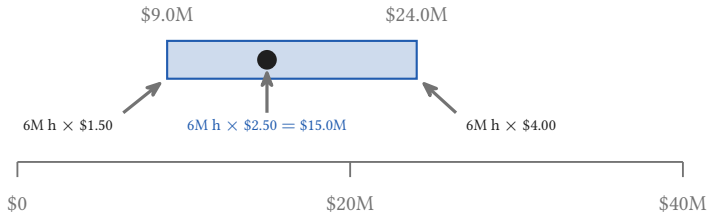
Six million hours, priced at cloud rates

The 70B run,
6 million hours

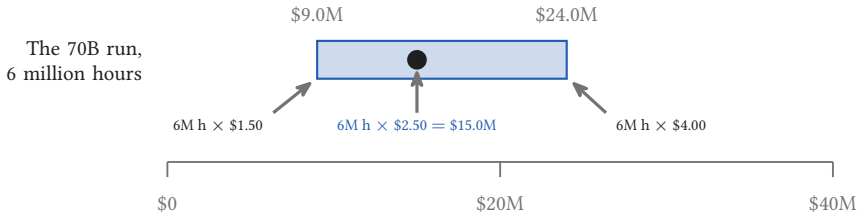


Six million hours, priced at cloud rates

The 70B run,
6 million hours



Six million hours, priced at cloud rates



The \$1.50 to \$4.00 band is an H100 rental survey, November 2025.

The same hours, in gigawatt hours

8.85 million GPU hours
(90 days $\times$ 4,096 GPUs)

The same hours, in gigawatt hours

8.85 million GPU hours
(90 days $\times$ 4,096 GPUs)

$\times$

560 W per module

The same hours, in gigawatt hours

8.85 million GPU hours
(90 days $\times$ 4,096 GPUs)

$\times$

560 W per module

$=$

4.95 GWh

The same hours, in gigawatt hours

8.85 million GPU hours
(90 days $\times$ 4,096 GPUs)

$\times$

560 W per module

$=$

4.95 GWh

4.95

3.4

8.6



The same hours, in gigawatt hours

8.85 million GPU hours
(90 days $\times$ 4,096 GPUs)

$\times$

560 W per module

$=$

4.95 GWh

4.95

3.4

8.6



- About 1,100 Swiss households for a year, at 4,500 kWh each

The same hours, in gigawatt hours

8.85 million GPU hours
(90 days × 4,096 GPUs)

×

560 W per module

=

4.95 GWh

4.95

3.4

8.6



- About 1,100 Swiss households for a year, at 4,500 kWh each

Range 750 to 1,900 households, over four sourced module figures: **560 W** tech report, [arXiv:2509.14233](https://arxiv.org/abs/2509.14233); **660 W** CSCS cap, docs.cscs.ch/running/slurm; **855 W** from the TOP500 run, top500.org/system/180259; datasheet band 450 to 1,000 W.

The same hours, in gigawatt hours

8.85 million GPU hours
(90 days × 4,096 GPUs)

×

560 W per module

=

4.95 GWh

4.95

3.4



8.6



- About 1,100 Swiss households for a year, at 4,500 kWh each

Range 750 to 1,900 households, over four sourced module figures: **560 W** tech report, [arXiv:2509.14233](https://arxiv.org/abs/2509.14233); **660 W** CSCS cap, docs.cscs.ch/running/slurm; **855 W** from the TOP500 run, top500.org/system/180259; datasheet band 450 to 1,000 W.

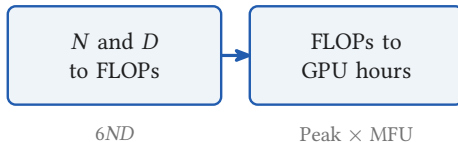
Household 4,500 kWh a year is ElCom's H4 profile. CSCS publish a PUE below 1.2, cooling from Lake Lugano (cscs.ch).

What you can price at 09:00

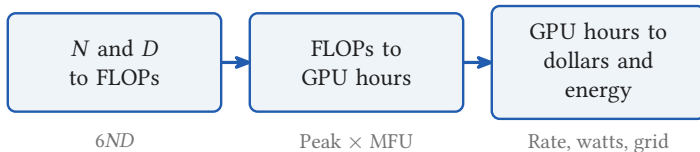
N and D
to FLOPs

$6ND$

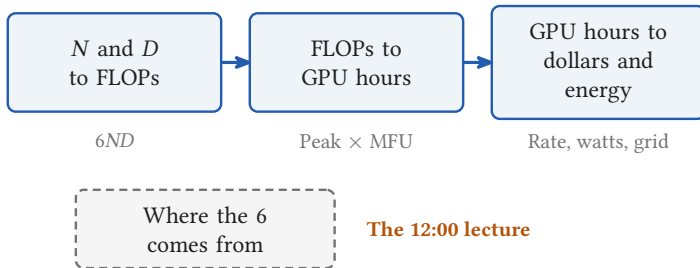
What you can price at 09:00



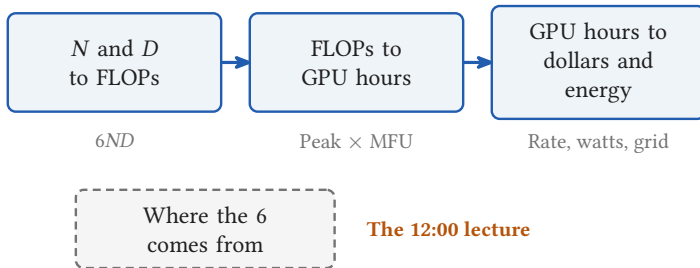
What you can price at 09:00



What you can price at 09:00

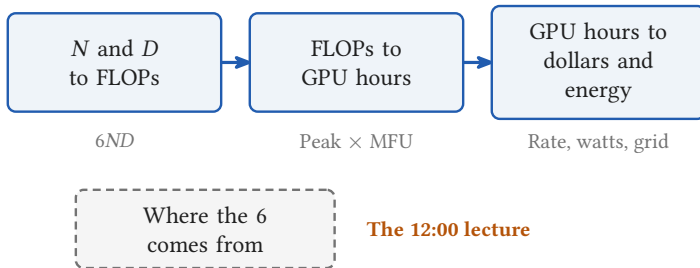


What you can price at 09:00



- You can now price a training run on the back of an envelope

What you can price at 09:00



- You can now price a training run on the back of an envelope
- At 09:00 you build the envelope